Training a network, in practice

AI Methods for Science – Part I – Week 1, Thursday

Yohai Bar-Sinai

October 22, 2026

Where we are

1. A flexible function f_θ

the MLP: 1-64-64-1, then 3072-512-256-10

2. A cost function $L(\theta)$

from a likelihood: MSE, cross-entropy

3. An optimizer

SGD, momentum, Adam

Today:

1. **Losses from likelihoods**: regression, classification, why cross-entropy
2. **Optimizers**: why not full-batch GD, the minibatch, the learning rate, momentum, Adam
3. **Tooling**: autograd, `nn`, `torch.optim`, the five-line training loop
4. **First task**: fit a function, hands-on
5. **Training in practice**: CIFAR-10, data and normalization, the loop with validation, train / validation / test, overtraining
6. **Regularization**: weight decay
7. **Lab**: an MLP on CIFAR-10. The network and the loop are given; you run the experiments.

Losses from likelihoods:

which loss, and why

The loss is a likelihood: regression

Data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, ansatz f_θ. Which loss? Say what the data are: a model $p_\theta(y \mid \mathbf{x})$, and maximize its likelihood: $L(\theta) = -\frac{1}{n}\sum_i \log p_\theta(y_i \mid \mathbf{x}_i)$.

The loss is a likelihood: regression

Data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, ansatz f_θ. Which loss? Say what the data are: a model $p_\theta(y \mid \mathbf{x})$, and maximize its likelihood: $L(\theta) = -\frac{1}{n}\sum_i \log p_\theta(y_i \mid \mathbf{x}_i)$.

Gaussian noise, $y = f_\theta(\mathbf{x}) + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$:

$$-\log p_\theta(y \mid \mathbf{x}) = \frac{(y - f_\theta(\mathbf{x}))^2}{2\sigma^2} + \tfrac{1}{2}\log(2\pi\sigma^2) \quad \Rightarrow \quad L = \frac{1}{2\sigma^2}\underbrace{\frac{1}{n}\sum_i \big(y_i - f_\theta(\mathbf{x}_i)\big)^2}_{\text{MSE}} + \text{const.}$$

Least squares is maximum likelihood under Gaussian noise.

The loss is a likelihood: regression

Data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, ansatz f_θ. Which loss? Say what the data are: a model $p_\theta(y \mid \mathbf{x})$, and maximize its likelihood: $L(\theta) = -\frac{1}{n}\sum_i \log p_\theta(y_i \mid \mathbf{x}_i)$.

Gaussian noise, $y = f_\theta(\mathbf{x}) + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$:

$$-\log p_\theta(y \mid \mathbf{x}) = \frac{(y - f_\theta(\mathbf{x}))^2}{2\sigma^2} + \tfrac{1}{2}\log(2\pi\sigma^2) \quad \Rightarrow \quad L = \frac{1}{2\sigma^2}\underbrace{\frac{1}{n}\sum_i \big(y_i - f_\theta(\mathbf{x}_i)\big)^2}_{\text{MSE}} + \text{const.}$$

Least squares is maximum likelihood under Gaussian noise.

- **The floor.** $f^\star(\mathbf{x}) = \mathbb{E}[y \mid \mathbf{x}]$: $\mathbb{E}[(y-f)^2] = \mathbb{E}[(f^\star - f)^2] + \sigma^2 \geq \sigma^2$. Test MSE below σ^2 is impossible in expectation; training MSE below it is fitting the noise. The first task measures this: $\sigma^2 = 0.0625$.

The loss is a likelihood: regression

Data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, ansatz f_θ. Which loss? Say what the data are: a model $p_\theta(y \mid \mathbf{x})$, and maximize its likelihood: $L(\theta) = -\frac{1}{n}\sum_i \log p_\theta(y_i \mid \mathbf{x}_i)$.

Gaussian noise, $y = f_\theta(\mathbf{x}) + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$:

$$-\log p_\theta(y \mid \mathbf{x}) = \frac{(y - f_\theta(\mathbf{x}))^2}{2\sigma^2} + \tfrac{1}{2}\log(2\pi\sigma^2) \quad \Rightarrow \quad L = \frac{1}{2\sigma^2}\underbrace{\frac{1}{n}\sum_i \big(y_i - f_\theta(\mathbf{x}_i)\big)^2}_{\text{MSE}} + \text{const.}$$

Least squares is maximum likelihood under Gaussian noise.

- **The floor.** $f^\star(\mathbf{x}) = \mathbb{E}[y \mid \mathbf{x}]$: $\mathbb{E}[(y-f)^2] = \mathbb{E}[(f^\star - f)^2] + \sigma^2 \geq \sigma^2$. Test MSE below σ^2 is impossible in expectation; training MSE below it is fitting the noise. The first task measures this: $\sigma^2 = 0.0625$.
- **Change the noise, change the loss.** Laplace $\to |y - f|$ (median; robust). Poisson counts $\to f - y\log f$. $\mathbf{x}$-dependent noise: two outputs $f(\mathbf{x}), \sigma(\mathbf{x})$, $\ell = \frac{(y-f)^2}{2\sigma(\mathbf{x})^2} + \log\sigma(\mathbf{x})$.

The loss is a likelihood: classification

K classes, $y \in \{1, \ldots, K\}$. The network outputs K numbers, the **logits** $\mathbf{z} = f_\theta(\mathbf{x}) \in \mathbb{R}^K$:

$$p_k = \mathrm{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_j e^{z_j}} \qquad \text{a Boltzmann distribution, } E_k = -z_k,\ T = 1.$$

The loss is a likelihood: classification

K classes, $y \in \{1, \ldots, K\}$. The network outputs K numbers, the **logits** $\mathbf{z} = f_\theta(\mathbf{x}) \in \mathbb{R}^K$:

$$p_k = \mathrm{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_j e^{z_j}} \qquad \text{a Boltzmann distribution, } E_k = -z_k,\ T = 1.$$

- $\mathbf{z} \to \mathbf{z} + const$ leaves $\mathbf{p}$ unchanged, like (free) energies. $z_k - z_j = \log(p_k/p_j)$: logits are log-odds.

The loss is a likelihood: classification

K classes, $y \in \{1, \dots, K\}$. The network outputs K numbers, the **logits** $\mathbf{z} = f_\theta(\mathbf{x}) \in \mathbb{R}^K$:

$$p_k = \mathrm{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_j e^{z_j}} \qquad \text{a Boltzmann distribution, } E_k = -z_k,\ T = 1.$$

- $\mathbf{z} \to \mathbf{z} + const$ leaves $\mathbf{p}$ unchanged, like (free) energies. $z_k - z_j = \log(p_k/p_j)$: logits are log-odds.
- Negative log-likelihood of the correct class:

$$\ell(\mathbf{z}, y) = -\log p_y = -z_y + \log \sum_j e^{z_j} = E_y - F\ , \qquad F = -\log Z$$

The **cross-entropy** $-\sum_k y_k \log p_k$ with one-hot $\mathbf{y}$; averaged over the data, $\mathrm{KL}(\hat{p} \,\|\, p_\theta) + \text{const}$.

The loss is a likelihood: classification

K classes, $y \in \{1, \dots, K\}$. The network outputs K numbers, the **logits** $\mathbf{z} = f_\theta(\mathbf{x}) \in \mathbb{R}^K$:

$$p_k = \mathrm{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_j e^{z_j}} \qquad \text{a Boltzmann distribution, } E_k = -z_k,\ T = 1.$$

- $\mathbf{z} \to \mathbf{z} + const$ leaves $\mathbf{p}$ unchanged, like (free) energies. $z_k - z_j = \log(p_k/p_j)$: logits are log-odds.
- Negative log-likelihood of the correct class:

$$\ell(\mathbf{z}, y) = -\log p_y = -z_y + \log \sum_j e^{z_j} = E_y - F\ , \qquad F = -\log Z$$

  The **cross-entropy** $-\sum_k y_k \log p_k$ with one-hot $\mathbf{y}$; averaged over the data, $\mathrm{KL}(\hat{p} \,\|\, p_\theta) + \text{const}$.
- Gradient at the output:

$$\frac{\partial \ell}{\partial z_k} = -\delta_{ky} + \frac{e^{z_k}}{Z} = p_k - y_k, \qquad \frac{\partial \ell}{\partial \mathbf{z}} = \mathbf{p} - \mathbf{y}.$$

The loss is a likelihood: classification

K classes, $y \in \{1, \dots, K\}$. The network outputs K numbers, the **logits** $\mathbf{z} = f_\theta(\mathbf{x}) \in \mathbb{R}^K$:

$$p_k = \mathrm{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_j e^{z_j}} \qquad \text{a Boltzmann distribution, } E_k = -z_k,\ T = 1.$$

- $\mathbf{z} \to \mathbf{z} + const$ leaves $\mathbf{p}$ unchanged, like (free) energies. $z_k - z_j = \log(p_k/p_j)$: logits are log-odds.
- Negative log-likelihood of the correct class:

$$\ell(\mathbf{z}, y) = -\log p_y = -z_y + \log \sum_j e^{z_j} = E_y - F\ , \qquad F = -\log Z$$

  The **cross-entropy** $-\sum_k y_k \log p_k$ with one-hot $\mathbf{y}$; averaged over the data, $\mathrm{KL}(\hat{p} \,\|\, p_\theta) + \text{const}$.
- Gradient at the output:

$$\frac{\partial \ell}{\partial z_k} = -\delta_{ky} + \frac{e^{z_k}}{Z} = p_k - y_k, \qquad \frac{\partial \ell}{\partial \mathbf{z}} = \mathbf{p} - \mathbf{y}.$$

- **Warning:** Do not implement $\log \sum_j e^{z_j}$ naively: e^z overflows and $\log(0) = -\infty$ on underflow. Use `torch.logsumexp` or `nn.CrossEntropyLoss` does this.

Optimizers:

from the gradient to the update

Gradient descent, and why it is not an option as written

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t), \qquad L(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\theta)$$

Gradient descent, and why it is not an option as written

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t), \qquad L(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\theta)$$

- **Why first order at all.** $N = \dim \theta \sim 10^6$–$10^9$. Hessian: N^2 entries, Newton is out. Zero order (finite differences, Nelder–Mead, evolution): $O(N)$ loss evaluations per gradient's worth of information. The gradient: ≈ 3 forward passes, independent of N (Monday).

Gradient descent, and why it is not an option as written

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t), \qquad L(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\theta)$$

- **Why first order at all.** $N = \dim \theta \sim 10^6$–$10^9$. Hessian: N^2 entries, Newton is out. Zero order (finite differences, Nelder–Mead, evolution): $O(N)$ loss evaluations per gradient's worth of information. The gradient: ≈ 3 forward passes, independent of N (Monday).
- **Why not the full gradient.** One step is a forward and a backward pass over all n examples ($45{,}000$ here; 10^6–10^9 in general). The backward pass keeps every activation of every example; a GPU holds 10^2–10^3 images, not the data set.

Gradient descent, and why it is not an option as written

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t), \qquad L(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\theta)$$

- **Why first order at all.** $N = \dim \theta \sim 10^6$–$10^9$. Hessian: N^2 entries, Newton is out. Zero order (finite differences, Nelder–Mead, evolution): $O(N)$ loss evaluations per gradient's worth of information. The gradient: ≈ 3 forward passes, independent of N (Monday).
- **Why not the full gradient.** One step is a forward and a backward pass over all n examples ($45{,}000$ here; 10^6–10^9 in general). The backward pass keeps every activation of every example; a GPU holds 10^2–10^3 images, not the data set.
- **And it would be wasted.** Nearby examples carry nearly the same gradient information: a few hundred examples already give the direction. A step from all n costs as much as hundreds of steps from such subsets, and is not hundreds of times better.

Gradient descent, and why it is not an option as written

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t), \qquad L(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\theta)$$

- **Why first order at all.** $N = \dim \theta \sim 10^6$–$10^9$. Hessian: N^2 entries, Newton is out. Zero order (finite differences, Nelder–Mead, evolution): $O(N)$ loss evaluations per gradient's worth of information. The gradient: ≈ 3 forward passes, independent of N (Monday).
- **Why not the full gradient.** One step is a forward and a backward pass over all n examples ($45{,}000$ here; 10^6–10^9 in general). The backward pass keeps every activation of every example; a GPU holds 10^2–10^3 images, not the data set.
- **And it would be wasted.** Nearby examples carry nearly the same gradient information: a few hundred examples already give the direction. A step from all n costs as much as hundreds of steps from such subsets, and is not hundreds of times better.

Key point

Estimate the gradient from a few examples, step, repeat: stochastic gradient descent.

Stochastic gradient descent (SGD): the minibatch

Minibatch $\mathcal{B} \subset \{1, \dots, n\}$, $|\mathcal{B}| = B$, drawn at random:

$$\hat{g} = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla \ell_i(\theta), \qquad \mathbb{E}[\hat{g}] = \nabla L, \qquad \mathrm{Cov}[\hat{g}] = \frac{C}{B},$$

C the covariance of the per-example gradients. Unbiased; noise amplitude $\propto 1/\sqrt{B}$; cost $\propto B$.

Stochastic gradient descent (SGD): the minibatch

Minibatch $\mathcal{B} \subset \{1, \dots, n\}$, $|\mathcal{B}| = B$, drawn at random:

$$\hat{g} = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla \ell_i(\theta), \qquad \mathbb{E}[\hat{g}] = \nabla L, \qquad \mathrm{Cov}[\hat{g}] = \frac{C}{B},$$

C the covariance of the per-example gradients. Unbiased; noise amplitude $\propto 1/\sqrt{B}$; cost $\propto B$.

- **Jargon.** Batch size B: 32–512, set by memory. **Epoch**: one pass over the data, n/B steps. **Shuffle** every epoch.

Stochastic gradient descent (SGD): the minibatch

Minibatch $\mathcal{B} \subset \{1, \dots, n\}$, $|\mathcal{B}| = B$, drawn at random:

$$\hat{g} = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla \ell_i(\theta), \qquad \mathbb{E}[\hat{g}] = \nabla L, \qquad \mathrm{Cov}[\hat{g}] = \frac{C}{B},$$

C the covariance of the per-example gradients. Unbiased; noise amplitude $\propto 1/\sqrt{B}$; cost $\propto B$.

- **Jargon.** Batch size B: 32–512, set by memory. **Epoch**: one pass over the data, n/B steps. **Shuffle** every epoch.
- The update is the same line, $\theta \leftarrow \theta - \eta\, \hat{g}$. Today's first task is full batch ($B = n = 200$): gradient descent, no noise.

Stochastic gradient descent (SGD): the minibatch

Minibatch $\mathcal{B} \subset \{1, \dots, n\}$, $|\mathcal{B}| = B$, drawn at random:

$$\hat{g} = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla \ell_i(\theta), \qquad \mathbb{E}[\hat{g}] = \nabla L, \qquad \mathrm{Cov}[\hat{g}] = \frac{C}{B},$$

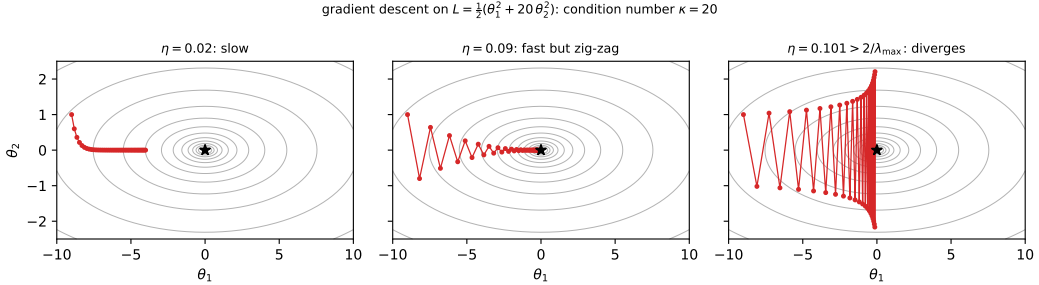
C the covariance of the per-example gradients. Unbiased; noise amplitude $\propto 1/\sqrt{B}$; cost $\propto B$.

- **Jargon.** Batch size B: 32–512, set by memory. **Epoch**: one pass over the data, n/B steps. **Shuffle** every epoch.
- The update is the same line, $\theta \leftarrow \theta - \eta\, \hat{g}$. Today's first task is full batch ($B = n = 200$): gradient descent, no noise.

Physicist's translation

Gradient flow $\dot{\theta} = -\nabla L$ is overdamped dynamics in the potential L; GD is its Euler step. Minibatch noise makes it Langevin dynamics at an "effective temperature" $\propto \eta\, C/B$ (kinda). The noise leaves narrow minima and saddles.

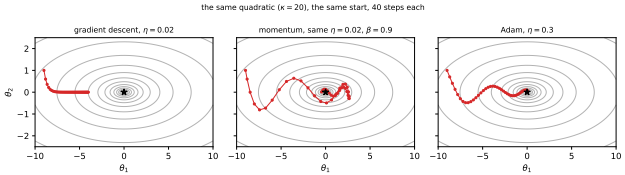
The learning rate and the condition number



Quadratic with Hessian eigenvalues λ_i: each component contracts by $(1 - \eta\lambda_i)$ per step. Stable iff $\eta < 2/\lambda_{\max}$; the slowest direction takes $\sim \kappa \log(1/\epsilon)$ steps, $\kappa = \lambda_{\max}/\lambda_{\min}$. The sharpest direction sets η; the flattest sets the time.

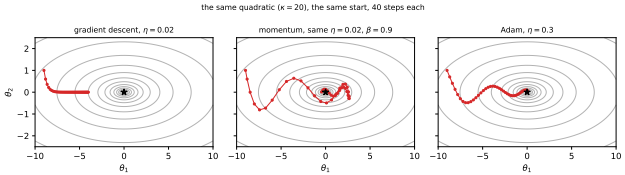
The learning rate is the one hyperparameter you always have to choose. Too large: `nan`. Too small: nothing moves. Rule of thumb for plain SGD: the largest η that does not blow up, then halve it. The lab's E2 is this sweep on the real problem.

Momentum and Adam



Momentum: a heavy ball. The step is a running average of past gradients: $v \leftarrow \beta v + g$, $\theta \leftarrow \theta - \eta v$.

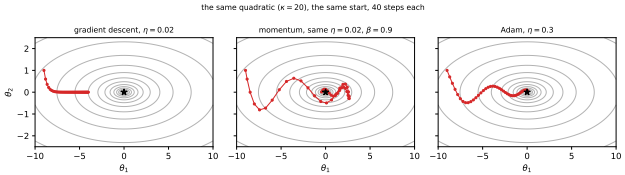
Momentum and Adam



Momentum: a heavy ball. The step is a running average of past gradients: $v \leftarrow \beta v + g$, $\theta \leftarrow \theta - \eta v$.

- Across the valley the gradient alternates sign: the average cancels the zig-zag.
- Along the valley it keeps its sign: the speed builds up to $1/(1-\beta)$ plain steps ($10\times$ for $\beta = 0.9$); it overshoots a little.

Momentum and Adam



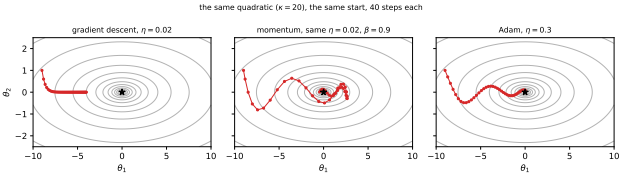
Momentum: a heavy ball. The step is a running average of past gradients: $v \leftarrow \beta v + g$, $\theta \leftarrow \theta - \eta v$.

- Across the valley the gradient alternates sign: the average cancels the zig-zag.
- Along the valley it keeps its sign: the speed builds up to $1/(1-\beta)$ plain steps ($10\times$ for $\beta = 0.9$); it overshoots a little.

Adam: a step size per parameter. Each gradient component is divided by its own recent size: $\theta_j \leftarrow \theta_j - \eta \langle g_j \rangle / \sqrt{\langle g_j^2 \rangle}$.

- A consistent gradient moves θ_j by about η per step, whether it is 10^{-6} or 10^{2}; a noisy one moves it less.

Momentum and Adam



Momentum: a heavy ball. The step is a running average of past gradients: $v \leftarrow \beta v + g$, $\theta \leftarrow \theta - \eta v$.

- Across the valley the gradient alternates sign: the average cancels the zig-zag.
- Along the valley it keeps its sign: the speed builds up to $1/(1-\beta)$ plain steps ($10\times$ for $\beta = 0.9$); it overshoots a little.

Adam: a step size per parameter. Each gradient component is divided by its own recent size: $\theta_j \leftarrow \theta_j - \eta \langle g_j \rangle / \sqrt{\langle g_j^2 \rangle}$.

- A consistent gradient moves θ_j by about η per step, whether it is 10^{-6} or 10^{2}; a noisy one moves it less.
- Default $\eta = 10^{-3}$; the usual first choice. Today: Adam on the toy task, plain SGD in the lab. The why: Monday.

Warning. Low-dimensional pictures like these are sometimes misleading in 10^6 dimensions: there almost every critical point is a saddle, the noise matters, and what works is partly empirical.

Tooling:

the five lines, in PyTorch

Tools

- **PyTorch**: a tape; `loss.backward()`. What we use.

Tools

- **PyTorch**: a tape; `loss.backward()`. What we use.
- **JAX**: differentiation as a *function transformation*: `grad`, `vmap`, `jit`, composable.

```
import jax, jax.numpy as jnp
f = lambda x: jnp.sin(x[0]*x[1]) + jnp.exp(-x[1]**2)

g = jax.grad(f)(x) # exact gradient
H = jax.hessian(f)(x) # forward-over-reverse
gs = jax.vmap(jax.grad(f))(xs) # batched, no loop
fg = jax.jit(jax.grad(f)) # compile once
```

Tools

- **PyTorch**: a tape; `loss.backward()`. What we use.
- **JAX**: differentiation as a *function transformation*: `grad`, `vmap`, `jit`, composable.
- **Julia**: Zygote, Enzyme (also C/C++/Fortran).

```
import jax, jax.numpy as jnp
f = lambda x: jnp.sin(x[0]*x[1]) + jnp.exp(-x[1]**2)

g = jax.grad(f)(x) # exact gradient
H = jax.hessian(f)(x) # forward-over-reverse
gs = jax.vmap(jax.grad(f))(xs) # batched, no loop
fg = jax.jit(jax.grad(f)) # compile once
```

Autograd: Monday's example, in five lines

```
import torch, math
a, b, c, d = (torch.tensor(v, requires_grad=True)
              for v in (2.0, math.pi, 1.0, 2.0))
f = a * torch.sin(b / (c + d)) # forward, recorded
f.backward() # backward, one pass
print(f.item(), a.grad, b.grad, c.grad, d.grad)
# 1.7321 0.8660 0.3333 -0.3491 -0.3491
```

Autograd: Monday's example, in five lines

```
import torch, math
a, b, c, d = (torch.tensor(v, requires_grad=True)
              for v in (2.0, math.pi, 1.0, 2.0))
f = a * torch.sin(b / (c + d)) # forward, recorded
f.backward() # backward, one pass
print(f.item(), a.grad, b.grad, c.grad, d.grad)
# 1.7321 0.8660 0.3333 -0.3491 -0.3491
```

- `requires_grad=True` marks a leaf of the graph.

Autograd: Monday's example, in five lines

```
import torch, math
a, b, c, d = (torch.tensor(v, requires_grad=True)
              for v in (2.0, math.pi, 1.0, 2.0))
f = a * torch.sin(b / (c + d)) # forward, recorded
f.backward() # backward, one pass
print(f.item(), a.grad, b.grad, c.grad, d.grad)
# 1.7321 0.8660 0.3333 -0.3491 -0.3491
```

- `requires_grad=True` marks a leaf of the graph.
- Every operation on such a tensor records itself and its local derivative (the tape).

Autograd: Monday's example, in five lines

```
import torch, math
a, b, c, d = (torch.tensor(v, requires_grad=True)
              for v in (2.0, math.pi, 1.0, 2.0))
f = a * torch.sin(b / (c + d)) # forward, recorded
f.backward() # backward, one pass
print(f.item(), a.grad, b.grad, c.grad, d.grad)
# 1.7321 0.8660 0.3333 -0.3491 -0.3491
```

- `requires_grad=True` marks a leaf of the graph.
- Every operation on such a tensor records itself and its local derivative (the tape).
- `.backward()` runs Monday's backward pass from this node; each leaf gets `.grad`.

Autograd: Monday's example, in five lines

```
import torch, math
a, b, c, d = (torch.tensor(v, requires_grad=True)
              for v in (2.0, math.pi, 1.0, 2.0))
f = a * torch.sin(b / (c + d)) # forward, recorded
f.backward() # backward, one pass
print(f.item(), a.grad, b.grad, c.grad, d.grad)
# 1.7321 0.8660 0.3333 -0.3491 -0.3491
```

- `requires_grad=True` marks a leaf of the graph.
- Every operation on such a tensor records itself and its local derivative (the tape).
- `.backward()` runs Monday's backward pass from this node; each leaf gets `.grad`.
- `.grad` *accumulates* across calls. Hence `zero_grad()` in every training step.

$-\pi/9 = -0.3491$: the same numbers as the slide on Monday.

A model: nn.Sequential

```
import torch, torch.nn as nn
model = nn.Sequential(nn.Linear(1, 64), nn.Tanh(),
                      nn.Linear(64, 64), nn.Tanh(),
                      nn.Linear(64, 1))

x = torch.rand(200, 1) # (batch, features)
model(x).shape # torch.Size([200, 1])
sum(p.numel() for p in model.parameters()) # 4353
```

A model: `nn.Sequential`

```
import torch, torch.nn as nn
model = nn.Sequential(nn.Linear(1, 64), nn.Tanh(),
                      nn.Linear(64, 64), nn.Tanh(),
                      nn.Linear(64, 1))

x = torch.rand(200, 1) # (batch, features)
model(x).shape # torch.Size([200, 1])
sum(p.numel() for p in model.parameters()) # 4353
```

- `nn.Linear(n,m)` is $\mathbf{W}\mathbf{x} + \mathbf{b}$, $\mathbf{W} \in \mathbb{R}^{m \times n}$; $\mathbf{W}, \mathbf{b}$ registered as parameters and initialized.

A model: `nn.Sequential`

```
import torch, torch.nn as nn
model = nn.Sequential(nn.Linear(1, 64), nn.Tanh(),
                      nn.Linear(64, 64), nn.Tanh(),
                      nn.Linear(64, 1))

x = torch.rand(200, 1) # (batch, features)
model(x).shape # torch.Size([200, 1])
sum(p.numel() for p in model.parameters()) # 4353
```

- `nn.Linear(n,m)` is $\mathbf{W}\mathbf{x} + \mathbf{b}$, $\mathbf{W} \in \mathbb{R}^{m \times n}$; $\mathbf{W}, \mathbf{b}$ registered as parameters and initialized.
- `model.parameters()` yields all of them: that is θ.

A model: `nn.Sequential`

```
import torch, torch.nn as nn
model = nn.Sequential(nn.Linear(1, 64), nn.Tanh(),
                      nn.Linear(64, 64), nn.Tanh(),
                      nn.Linear(64, 1))

x = torch.rand(200, 1) # (batch, features)
model(x).shape # torch.Size([200, 1])
sum(p.numel() for p in model.parameters()) # 4353
```

- `nn.Linear(n,m)` is $\mathbf{W}\mathbf{x} + \mathbf{b}$, $\mathbf{W} \in \mathbb{R}^{m \times n}$; $\mathbf{W}, \mathbf{b}$ registered as parameters and initialized.
- `model.parameters()` yields all of them: that is θ.
- The first dimension is the batch.

Losses and optimizers

`nn.MSELoss()`	$\frac{1}{n}\sum_i \left(f_\theta(\mathbf{x}_i) - y_i\right)^2$

Losses and optimizers

`nn.MSELoss()`	$\frac{1}{n}\sum_i \left(f_\theta(\mathbf{x}_i) - y_i\right)^2$
`nn.CrossEntropyLoss()`	$-\log \mathrm{softmax}(\mathbf{z})_y$; takes the logits $\mathbf{z}$, not probabilities

Losses and optimizers

`nn.MSELoss()`	$\frac{1}{n}\sum_i \left(f_\theta(\mathbf{x}_i) - y_i\right)^2$
`nn.CrossEntropyLoss()`	$-\log \mathrm{softmax}(\mathbf{z})_y$; takes the logits $\mathbf{z}$, not probabilities
`torch.optim.SGD(params, lr)`	the plain step along the gradient; one knob, the learning rate

Losses and optimizers

`nn.MSELoss()`	$\frac{1}{n}\sum_i \left(f_\theta(\mathbf{x}_i) - y_i\right)^2$
`nn.CrossEntropyLoss()`	$-\log \mathrm{softmax}(\mathbf{z})_y$; takes the logits $\mathbf{z}$, not probabilities
`torch.optim.SGD(params, lr)`	the plain step along the gradient; one knob, the learning rate
`SGD(params, lr, momentum=0.9)`	the same, with a running average of the gradient: the heavy ball

Losses and optimizers

`nn.MSELoss()`	$\frac{1}{n}\sum_i \big(f_\theta(\mathbf{x}_i) - y_i\big)^2$
`nn.CrossEntropyLoss()`	$-\log \mathrm{softmax}(\mathbf{z})_y$; takes the logits $\mathbf{z}$, not probabilities
`torch.optim.SGD(params, lr)`	the plain step along the gradient; one knob, the learning rate
`SGD(params, lr, momentum=0.9)`	the same, with a running average of the gradient: the heavy ball
`torch.optim.Adam(params, lr=1e-3)`	a step size per parameter; the usual first choice

Losses and optimizers

`nn.MSELoss()`	$\frac{1}{n}\sum_i \left(f_\theta(\mathbf{x}_i) - y_i\right)^2$
`nn.CrossEntropyLoss()`	$-\log \mathrm{softmax}(\mathbf{z})_y$; takes the logits $\mathbf{z}$, not probabilities
`torch.optim.SGD(params, lr)`	the plain step along the gradient; one knob, the learning rate
`SGD(params, lr, momentum=0.9)`	the same, with a running average of the gradient: the heavy ball
`torch.optim.Adam(params, lr=1e-3)`	a step size per parameter; the usual first choice

g is `p.grad` for each parameter `p`, filled by `loss.backward()`; `optimizer.step()` applies the update. `params` = `model.parameters()`.

The training loop: five lines

```
x = torch.rand(200, 1) # (200, 1)
noise = 0.25 * torch.randn(200, 1)
y = torch.sin(2 * torch.pi * x) + noise

optimizer = torch.optim.Adam(model.parameters(),
                             lr=1e-2)
loss_fn = nn.MSELoss()

for step in range(2000):
    optimizer.zero_grad()
    loss = loss_fn(model(x), y)
    loss.backward()
    optimizer.step()
```

The training loop: five lines

```
x = torch.rand(200, 1) # (200, 1)
noise = 0.25 * torch.randn(200, 1)
y = torch.sin(2 * torch.pi * x) + noise

optimizer = torch.optim.Adam(model.parameters(),
                             lr=1e-2)
loss_fn = nn.MSELoss()

for step in range(2000):
    optimizer.zero_grad()
    loss = loss_fn(model(x), y)
    loss.backward()
    optimizer.step()
```

`for step` — full batch: all 200 points every step

The training loop: five lines

```
x = torch.rand(200, 1) # (200, 1)
noise = 0.25 * torch.randn(200, 1)
y = torch.sin(2 * torch.pi * x) + noise

optimizer = torch.optim.Adam(model.parameters(),
                             lr=1e-2)
loss_fn = nn.MSELoss()

for step in range(2000):
    optimizer.zero_grad()
    loss = loss_fn(model(x), y)
    loss.backward()
    optimizer.step()
```

`for step`	full batch: all 200 points every step
`zero_grad()`	`.grad` accumulates; clear it

The training loop: five lines

```
x = torch.rand(200, 1) # (200, 1)
noise = 0.25 * torch.randn(200, 1)
y = torch.sin(2 * torch.pi * x) + noise

optimizer = torch.optim.Adam(model.parameters(),
                             lr=1e-2)
loss_fn = nn.MSELoss()

for step in range(2000):
    optimizer.zero_grad()
    loss = loss_fn(model(x), y)
    loss.backward()
    optimizer.step()
```

`for step`	full batch: all 200 points every step
`zero_grad()`	`.grad` accumulates; clear it
`model(x)`	forward pass; graph recorded
`loss_fn`	$\frac{1}{n}\sum_i (f_\theta(x_i) - y_i)^2$

The training loop: five lines

```
x = torch.rand(200, 1) # (200, 1)
noise = 0.25 * torch.randn(200, 1)
y = torch.sin(2 * torch.pi * x) + noise

optimizer = torch.optim.Adam(model.parameters(),
                             lr=1e-2)
loss_fn = nn.MSELoss()

for step in range(2000):
    optimizer.zero_grad()
    loss = loss_fn(model(x), y)
    loss.backward()
    optimizer.step()
```

`for step`	full batch: all 200 points every step
`zero_grad()`	`.grad` accumulates; clear it
`model(x)`	forward pass; graph recorded
`loss_fn`	$\frac{1}{n}\sum_i (f_\theta(x_i) - y_i)^2$
`backward()`	$\partial\,\mathrm{loss}/\partial\theta$ into every `.grad`

The training loop: five lines

```
x = torch.rand(200, 1) # (200, 1)
noise = 0.25 * torch.randn(200, 1)
y = torch.sin(2 * torch.pi * x) + noise

optimizer = torch.optim.Adam(model.parameters(),
                             lr=1e-2)
loss_fn = nn.MSELoss()

for step in range(2000):
    optimizer.zero_grad()
    loss = loss_fn(model(x), y)
    loss.backward()
    optimizer.step()
```

`for step`	full batch: all 200 points every step
`zero_grad()`	`.grad` accumulates; clear it
`model(x)`	forward pass; graph recorded
`loss_fn`	$\frac{1}{n}\sum_i (f_\theta(x_i) - y_i)^2$
`backward()`	$\partial\,\mathrm{loss}/\partial\theta$ into every `.grad`
`step()`	the optimizer's update, from `.grad`

The training loop: five lines

```
x = torch.rand(200, 1) # (200, 1)
noise = 0.25 * torch.randn(200, 1)
y = torch.sin(2 * torch.pi * x) + noise

optimizer = torch.optim.Adam(model.parameters(),
                             lr=1e-2)
loss_fn = nn.MSELoss()

for step in range(2000):
    optimizer.zero_grad()
    loss = loss_fn(model(x), y)
    loss.backward()
    optimizer.step()
```

`for step`	full batch: all 200 points every step
`zero_grad()`	`.grad` accumulates; clear it
`model(x)`	forward pass; graph recorded
`loss_fn`	$\frac{1}{n}\sum_i (f_\theta(x_i) - y_i)^2$
`backward()`	$\partial\,\mathrm{loss}/\partial\theta$ into every `.grad`
`step()`	the optimizer's update, from `.grad`

Five lines are the whole algorithm. Everything else is data, model and plotting.

First task:

fit a function, hands-on

First task: fit a function

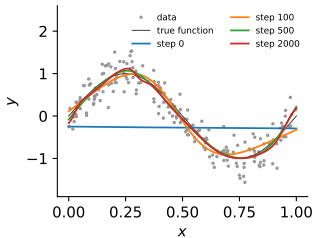
200 points, $x \in [0, 1]$ uniform, $y = \sin(2\pi x) + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, $\sigma = 0.25$. MLP 1–64–64–1, tanh; MSE; Adam; full batch.

1. Write the model, loss, optimizer, loop.
2. Print the loss every 200 steps.
3. Plot the fit over the data.

Then try:

- `--opt sgd` / `momentum` / `adam`
- `--hidden 1`: underfits; train MSE far above the noise floor
- `--steps 5000`: train MSE below the floor, test MSE rises: fitting the noise
- `--n 15 --steps 5000`: train MSE ≈ 0, test MSE ≈ 0.3: memorizing

Stretch: `--task spiral` (2D, two classes, cross-entropy).

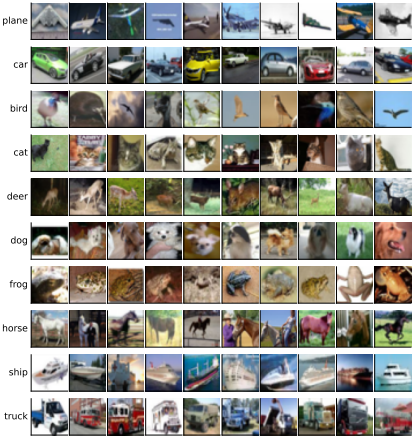


Reference solution: `demos/first_task.py`

Expect: the test MSE approaches the noise floor the script prints, and cannot go below it. Reference run (Adam, 2000 steps, 0.4 s on a laptop CPU): test MSE 0.058, floor 0.055 ($\sigma^2 = 0.0625$ in expectation).

Training in practice:
real data, real bookkeeping

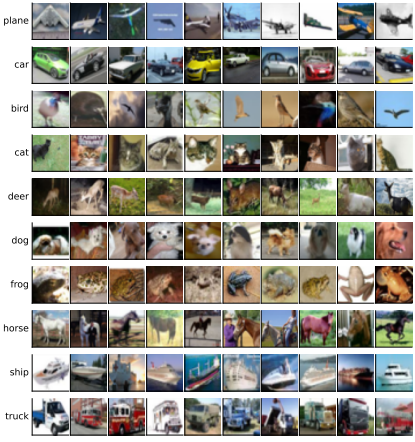
The task: CIFAR-10



Krizhevsky 2009. One row per class.

- $60{,}000$ colour images, $32 \times 32 \times 3$; 10 classes; $50{,}000$ train / $10{,}000$ test.

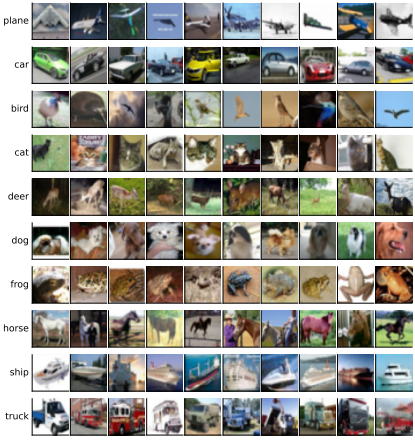
The task: CIFAR-10



Krizhevsky 2009. One row per class.

- $60{,}000$ colour images, $32 \times 32 \times 3$; 10 classes; $50{,}000$ train / $10{,}000$ test.
- Each image becomes a vector $\mathbf{x} \in \mathbb{R}^{3072}$. The MLP has no notion of a 2D grid; that gap is what architectures fix.

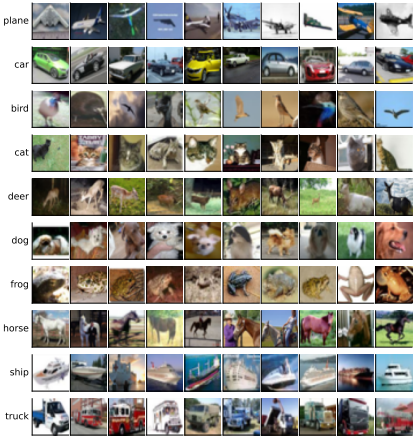
The task: CIFAR-10



Krizhevsky 2009. One row per class.

- $60{,}000$ colour images, $32 \times 32 \times 3$; 10 classes; $50{,}000$ train / $10{,}000$ test.
- Each image becomes a vector $\mathbf{x} \in \mathbb{R}^{3072}$. The MLP has no notion of a 2D grid; that gap is what architectures fix.
- The ansatz: $3072 \to 512 \to 256 \to 10$, ReLU between, logits out. $\#\theta = 3072 \cdot 512 + 512 + 512 \cdot 256 + 256 + 256 \cdot 10 + 10 = 1{,}707{,}274$.

The task: CIFAR-10



Krizhevsky 2009. One row per class.

- $60{,}000$ colour images, $32 \times 32 \times 3$; 10 classes; $50{,}000$ train / $10{,}000$ test.
- Each image becomes a vector $\mathbf{x} \in \mathbb{R}^{3072}$. The MLP has no notion of a 2D grid; that gap is what architectures fix.
- The ansatz: $3072 \to 512 \to 256 \to 10$, ReLU between, logits out. $\#\theta = 3072 \cdot 512 + 512 + 512 \cdot 256 + 256 + 256 \cdot 10 + 10 = 1{,}707{,}274$.
- Scale: chance 10%; a plain MLP $\sim 50\%$; a CNN $> 90\%$; humans $\approx 94\%$.

The training loop, with minibatches and validation

```
optimizer = torch.optim.SGD(model.parameters(),
                            lr=0.05)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(epochs):
    model.train()
    for x, y in train_loader:
        optimizer.zero_grad()
        logits = model(x)
        loss = loss_fn(logits, y)
        loss.backward()
        optimizer.step()

    model.eval()
    with torch.no_grad():
        for x, y in val_loader:
            ... # accuracy, loss on held-out data
```

What is new relative to the five lines:

`train_loader`	one step per minibatch of $B = 128$; an epoch $= 352$ steps
`loss_fn`	$\frac{1}{B}\sum_{i\in\mathcal{B}} \mathcal{L}_{\mathrm{CE}}$, from the logits
`SGD`, `step()`	$\theta \leftarrow \theta - \eta\,\hat{g}$
`eval()`, `no_grad()`	no graph, no activations stored; later also switches off dropout / BN statistics

The training loop, with minibatches and validation

```
optimizer = torch.optim.SGD(model.parameters(),
                            lr=0.05)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(epochs):
    model.train()
    for x, y in train_loader:
        optimizer.zero_grad()
        logits = model(x)
        loss = loss_fn(logits, y)
        loss.backward()
        optimizer.step()

    model.eval()
    with torch.no_grad():
        for x, y in val_loader:
            ... # accuracy, loss on held-out data
```

What is new relative to the five lines:

`train_loader`	one step per minibatch of $B = 128$; an epoch $= 352$ steps
`loss_fn`	$\frac{1}{B}\sum_{i\in\mathcal{B}} \mathcal{L}_{\mathrm{CE}}$, from the logits
`SGD`, `step()`	$\theta \leftarrow \theta - \eta\,\hat{g}$
`eval()`, `no_grad()`	no graph, no activations stored; later also switches off dropout / BN statistics

The inner four lines are the toy fit's, unchanged. Every other line of the script is bookkeeping and plotting.

The three failure modes, as the map of Part I

Failure	Symptom	Remedy
Optimization	training loss stays high	momentum, Adam, normalization, initialization
Approximation	the family cannot represent the answer	architectures as inductive bias
Generalization	training loss low, held-out loss high	splits, regularization, early stopping

The three failure modes, as the map of Part I

Failure	Symptom	Remedy
Optimization	training loss stays high	momentum, Adam, normalization, initialization
Approximation	the family cannot represent the answer	architectures as inductive bias
Generalization	training loss low, held-out loss high	splits, regularization, early stopping

Today's hands-on meets all three: the baseline trains (with the right η); it plateaus near 50% (an MLP on raw pixels); and it overfits if you let it run.

The three failure modes, as the map of Part I

Failure	Symptom	Remedy
Optimization	training loss stays high	momentum, Adam, normalization, initialization
Approximation	the family cannot represent the answer	architectures as inductive bias
Generalization	training loss low, held-out loss high	splits, regularization, early stopping

Today's hands-on meets all three: the baseline trains (with the right η); it plateaus near 50% (an MLP on raw pixels); and it overfits if you let it run.

The third one needs a measurement protocol before anything else. Next.

Train / validation / test



training: fits θ. The only set the gradient ever sees.

Train / validation / test



training: fits θ. The only set the gradient ever sees.

validation: every choice made by looking at a number – η, width, depth, batch size, when to stop – is made here.

Train / validation / test



training: fits θ. The only set the gradient ever sees.

validation: every choice made by looking at a number – η, width, depth, batch size, when to stop – is made here.

test: touched once, at the end, to report a number.

Train / validation / test



training: fits θ. The only set the gradient ever sees.

validation: every choice made by looking at a number – η, width, depth, batch size, when to stop – is made here.

test: touched once, at the end, to report a number.

Here: the standard $50{,}000/10{,}000$ partition, $5{,}000$ training images held out at random (fixed seed) as validation. Held-out error estimates the error on new data *only if* nothing was chosen by looking at it.

Train / validation / test



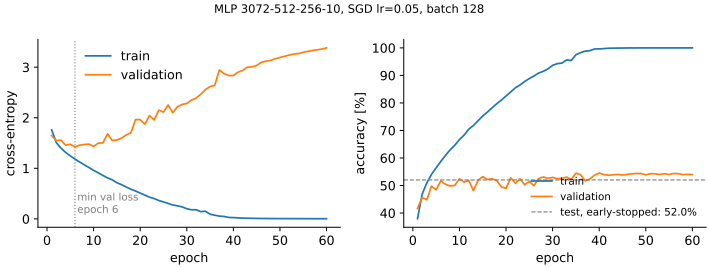
training: fits θ. The only set the gradient ever sees.

validation: every choice made by looking at a number – η, width, depth, batch size, when to stop – is made here.

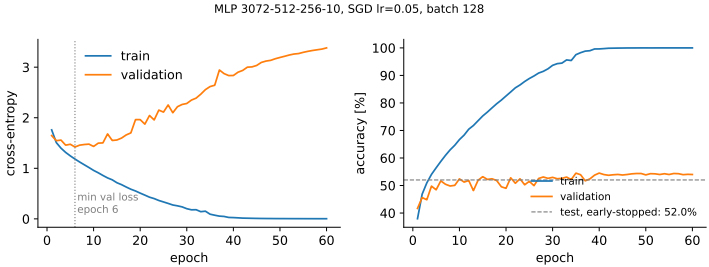
test: touched once, at the end, to report a number.

Here: the standard $50{,}000/10{,}000$ partition, $5{,}000$ training images held out at random (fixed seed) as validation. Held-out error estimates the error on new data *only if* nothing was chosen by looking at it. Note: in modern LLMs with tons of data, things are slightly different

What happens if you just keep training

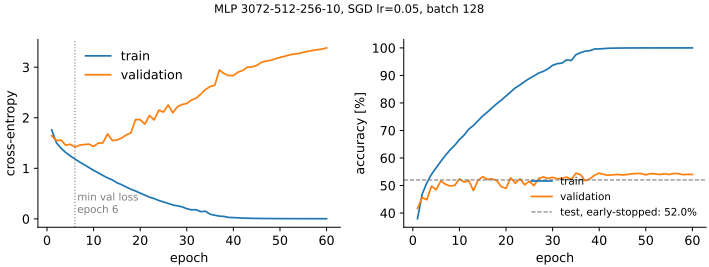


What happens if you just keep training



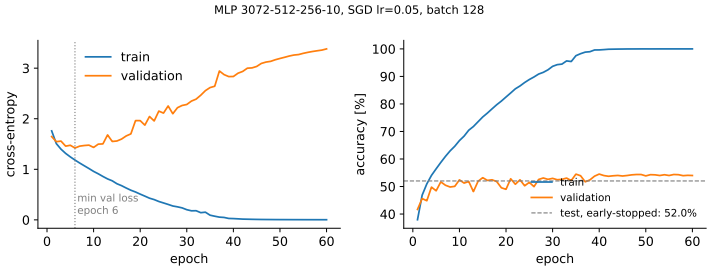
- training loss $\to 0$, training accuracy 100% by epoch 50: 1.7×10^6 parameters memorize $45{,}000$ examples

What happens if you just keep training



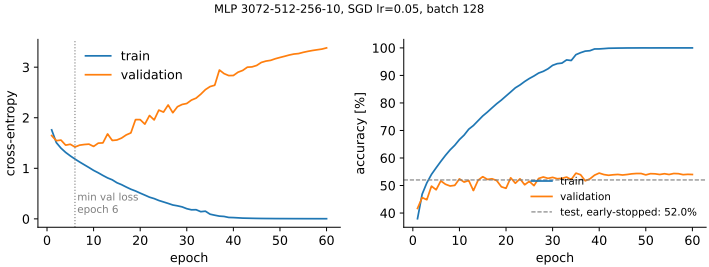
- training loss $\to 0$, training accuracy 100% by epoch 50: 1.7×10^6 parameters memorize $45{,}000$ examples
- validation loss is minimal at epoch 6 and then grows without bound: wrong answers, with growing confidence

What happens if you just keep training



- training loss $\to 0$, training accuracy 100% by epoch 50: 1.7×10^6 parameters memorize $45{,}000$ examples
- validation loss is minimal at epoch 6 and then grows without bound: wrong answers, with growing confidence
- validation *accuracy* still creeps up to $\approx 54\%$. Loss and accuracy disagree once the network is overconfident; decide which one you care about.

What happens if you just keep training



- training loss $\to 0$, training accuracy 100% by epoch 50: 1.7×10^6 parameters memorize $45{,}000$ examples
- validation loss is minimal at epoch 6 and then grows without bound: wrong answers, with growing confidence
- validation *accuracy* still creeps up to $\approx 54\%$. Loss and accuracy disagree once the network is overconfident; decide which one you care about.
- **early stopping**: keep the parameters of the best validation epoch, on that metric. The simplest regularizer there is; weight decay, dropout and augmentation are the others.

Regularization:

trading training loss for validation loss

Regularization: weight decay

The network can fit the training set (1.7×10^6 parameters, $45{,}000$ examples). Add a preference for small weights:

$$L_\lambda(\theta) = L(\theta) + \frac{\lambda}{2}\|\theta\|^2 \qquad \Rightarrow \qquad \theta \leftarrow (1 - \eta\lambda)\,\theta - \eta\,\nabla L(\theta).$$

Regularization: weight decay

The network can fit the training set (1.7×10^6 parameters, $45{,}000$ examples). Add a preference for small weights:

$$L_\lambda(\theta) = L(\theta) + \frac{\lambda}{2}\|\theta\|^2 \qquad \Rightarrow \qquad \theta \leftarrow (1 - \eta\lambda)\,\theta - \eta\,\nabla L(\theta).$$

- The weights *decay* toward zero by a factor $1 - \eta\lambda$ every step, unless the data push back. Code: `SGD(..., weight_decay=`λ`)`.

Regularization: weight decay

The network can fit the training set (1.7×10^6 parameters, $45{,}000$ examples). Add a preference for small weights:

$$L_\lambda(\theta) = L(\theta) + \frac{\lambda}{2}\|\theta\|^2 \qquad \Rightarrow \qquad \theta \leftarrow (1 - \eta\lambda)\,\theta - \eta\,\nabla L(\theta).$$

- The weights *decay* toward zero by a factor $1 - \eta\lambda$ every step, unless the data push back. Code: `SGD(..., weight_decay=`λ`)`.
- **It is a prior.** $\frac{\lambda}{2}\|\theta\|^2 = -\log p(\theta) + \text{const}$ for $\theta \sim \mathcal{N}(0, \lambda^{-1} I)$: the minimizer is the maximum a posteriori estimate. Loss from the likelihood, regularizer from the prior.

Regularization: weight decay

The network can fit the training set (1.7×10^6 parameters, $45{,}000$ examples). Add a preference for small weights:

$$L_\lambda(\theta) = L(\theta) + \frac{\lambda}{2}\|\theta\|^2 \qquad \Rightarrow \qquad \theta \leftarrow (1 - \eta\lambda)\,\theta - \eta\,\nabla L(\theta).$$

- The weights *decay* toward zero by a factor $1 - \eta\lambda$ every step, unless the data push back. Code: `SGD(..., weight_decay=`λ`)`.
- **It is a prior.** $\frac{\lambda}{2}\|\theta\|^2 = -\log p(\theta) + \text{const}$ for $\theta \sim \mathcal{N}(0, \lambda^{-1} I)$: the minimizer is the maximum a posteriori estimate. Loss from the likelihood, regularizer from the prior.
- Typical λ: 10^{-4}–5×10^{-4}. Too large: underfits, the ansatz is pulled to zero. With Adam use `AdamW`: the decay acts on θ directly, not through $1/\sqrt{\hat{v}}$.

Regularization: weight decay

The network can fit the training set (1.7×10^6 parameters, $45{,}000$ examples). Add a preference for small weights:

$$L_\lambda(\theta) = L(\theta) + \frac{\lambda}{2}\|\theta\|^2 \qquad \Rightarrow \qquad \theta \leftarrow (1 - \eta\lambda)\,\theta - \eta\,\nabla L(\theta).$$

- The weights *decay* toward zero by a factor $1 - \eta\lambda$ every step, unless the data push back. Code: `SGD(..., weight_decay=λ)`.
- **It is a prior.** $\frac{\lambda}{2}\|\theta\|^2 = -\log p(\theta) + \text{const}$ for $\theta \sim \mathcal{N}(0, \lambda^{-1} I)$: the minimizer is the maximum a posteriori estimate. Loss from the likelihood, regularizer from the prior.
- Typical λ: 10^{-4}–5×10^{-4}. Too large: underfits, the ansatz is pulled to zero. With Adam use `AdamW`: the decay acts on θ directly, not through $1/\sqrt{\hat{v}}$.

In the lab: the `weight_decay` key of `mlp_lab.py`, experiment E9.